

Chapter 05: Index Compression

https://drive.google.com/file/d/1oK5tx0pgfXPPCL_N5Bd3A9mjClpEoyTD/view

★ why compression?

- Use less disk space (saves money)
- Keep more stuff in memory (increases speed)
- Increase speed of transferring data from disk to memory
- read compressed data and decompress is faster than [read uncompressed data]
- Premise: Decompression algorithms must be fast
- In most cases, **retrieval system runs faster on compressed postings lists than on uncompressed postings lists.**

In IR systems, we mainly store:

1. **Dictionary** (terms → pointers to postings lists)
2. **Postings lists** (list of docIDs for each term)

Both take a LOT of space.

✓ 1. Compression for the Dictionary

“Make it small enough to keep in main memory”

Why?

- The dictionary is accessed for every query
- If it's stored in RAM, lookup is very fast
- If it's on disk, lookup becomes slow

So we compress the dictionary so that:

- ✓ It fits in main memory (RAM)
- ✓ Query processing becomes faster

✓ 2. Compression for the Postings Lists

Postings lists contain:

</> Code

```
term → [docID1, docID2, docID3, ...]
```

Each entry = a docID

For large collections:

- Millions of documents
- Billions of postings entries

That's HUGE.

› Why compress postings?

♦ (a) Reduce disk space needed

Without compression:

- Index becomes extremely large
- Storage cost increases

Each docID might take:

4 bytes (32 bits)

Compression reduces total index size.

♦ (b) Decrease time to read from disk

Disk I/O is slow.

If postings are compressed:

- Fewer bytes are read
- Less disk access
- Faster query processing

👉 Reading 100MB is slower than reading 20MB.

♦ (c) Large search engines keep postings in memory

Big systems (like Google) try to keep:

- Frequently accessed postings
- Important index parts

in RAM.

Compression helps:

- Fit more postings in memory



Acti

★ heap's law > > estimates **how vocabulary size grows** as the collection size increases.

> If we keep adding more documents, how many new unique words will appear?

In Heap's Law:

$$M = kT^b$$

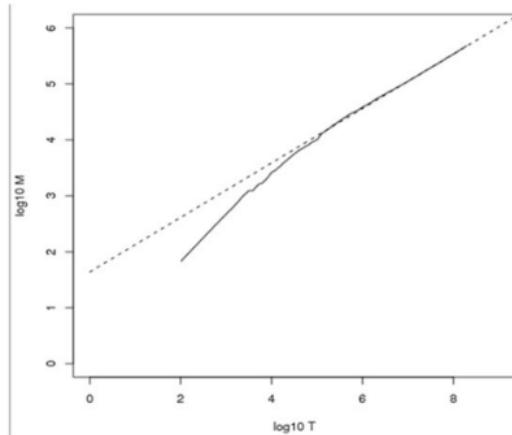
- M = vocabulary size (**unique terms**)
 - T = total number of word occurrences (tokens) – collection size
 - **k, b** = constants
 - $30 \leq k \leq 100$
 - $0.4 \leq b \leq 0.6$
 - $b \approx 0.5$ (usually)
-
- Vocabulary does NOT grow linearly.
 - as T increases, M also **increases** but at a **slower rate**
 - Because exponent **b < 1** → vocab growth **slows down** over time

This is just the **log version** of:

$$M = kT^b$$

Compare:

$$\log M = b \log T + \log k$$



◆ What Does the Graph Show?

- X-axis $\rightarrow \log_{10} T$ (collection size)
- Y-axis $\rightarrow \log_{10} M$ (vocabulary size)
- Dashed line $\rightarrow$ best fit

It becomes a **straight line** because of the **log transformation**.

◆ Important Insights (Second Slide)

1) k is variable

Vocabulary growth depends on:

- nature of documents
- Preprocessing steps

Example:

- News corpus $\rightarrow$ moderate growth
- Social media $\rightarrow$ fast growth (slang, typos, emojis)

2 Preprocessing affects growth

✓ **Case folding reduces vocabulary**

"Apple" and "apple" become same

✓ **Stemming reduces vocabulary**

"connect", "connected", "connecting" → "connect"

✗ **Including numbers increases vocabulary**

"2024", "2025", "1234"

✗ **Spelling errors increase vocabulary**

"recieve" vs "receive"

Heap's Law suggests: 🚀 **Vocabulary NEVER stops growing**

Even if you add millions of documents:

- New words keep appearing
- There is no fixed maximum vocabulary

But... **Growth slows down over time.**

★ **Zipf's law:**

- Explains the **long tail problem!**
- studies **how often words occur**
- In text collections:
 - A **few words** occur **very frequently** (head)
 - **Very many words** occur **very rarely** (tail)
- That long list of rare words = **long tail**.



Zipf's Law says:

$$cf_i \propto \frac{1}{i} \qquad cf_i = \frac{a}{i}$$

cf_i is collection frequency: the number of occurrences of the term t_i in the collection.

$a = \text{normalizing constant!}$

- As rank i increases:
 - Frequency drops
 - But it **never becomes zero**
 - So new rare words keep appearing
- This creates:
 - A steep **head**
 - A very long **tail** of low-frequency terms

That is the long-tail distribution.

◆ Zipf Consequences (Second Slide)

If:

Most frequent word occurs = cf_1

Then:

Second most frequent:

$$cf_2 = \frac{cf_1}{2}$$

Third most frequent:

$$cf_3 = \frac{cf_1}{3}$$

Fourth:

$$cf_4 = \frac{cf_1}{4}$$

And so on...

If:

$cf_1 = 10,000$

Then:

- $cf_2 = 5,000$
- $cf_3 \approx 3,333$
- $cf_4 = 2,500$
- $cf_{10} = 1,000$

See the pattern? It decreases smoothly.

◆ Why Log Graph Becomes Linear?

From:

$$cf_i = \frac{a}{i}$$

Take log:

$$\log cf_i = \log a - \log i$$

That is:

$$y = c - x$$

Which is a straight line.

So:

On a log-log plot → Zipf's law appears linear.

› Rank × Frequency ≈ Constant

$$R \times f \approx \text{constant}$$

Another way to write Zipf:

English:	Rank R	Word	Frequency f	$R \times f$
	10	he	877	8770
	20	but	410	8200
	30	be	294	8820
	800	friends	10	8000
	1000	family	8	8000

◆ Why This Is a “Problem” in IR

Because of the long tail:

- 1 Most vocabulary terms appear very few times
- 2 Dictionary size becomes huge
- 3 Many postings lists are extremely short
- 4 Index storage and memory management become challenging

Zipf tells us why this happens.

◆ Relation to Heap’s Law

Law	Explains
Zipf’s Law	Why there are many rare words (long tail)
Heap’s Law	Why vocabulary size keeps growing

Zipf → frequency distribution

Heap → vocabulary growth

Together they explain scalability issues in IR systems.

★ Dictionary Compression

- One of the primary factors in determining the response time of an IR system is the number of disk seeks necessary to process a query.
- If parts of the dictionary are on disk, then many more disk seeks are necessary in query evaluation.
- Thus, the main goal of compressing the dictionary → to fit it in main memory, or at least a large portion of it!

Fixed-Width Dictionary (Naive Method)

We store each term as:

- 20 bytes → term string
- 4 bytes → frequency
- 4 bytes → pointer to postings

> Avg english word ~ 8 chars

So we waste:

$$20 - 8 = 12 \text{ bytes per word}$$

So per term:

$$20 + 4 + 4 = 28 \text{ bytes}$$

For RCV1:

$$400,000 \times 28 = 11.2 \text{ MB}$$

So dictionary size = 11.2 MB

> dictionary as a string

We store: **term1term2term3term4...**

- One continuous string.
- Create a big string by concatenating strings in the dictionary
- store a pointer to each term's start position. [term ptr]

Step-by-Step New Storage Calculation

- ◆ Average term length = 8 bytes

Total string length:

$$400,000 \times 8 = 3.2 \text{ million bytes}$$

MB

◆ How many bits needed for term pointer?

We need to point inside a 3.2 million byte string.

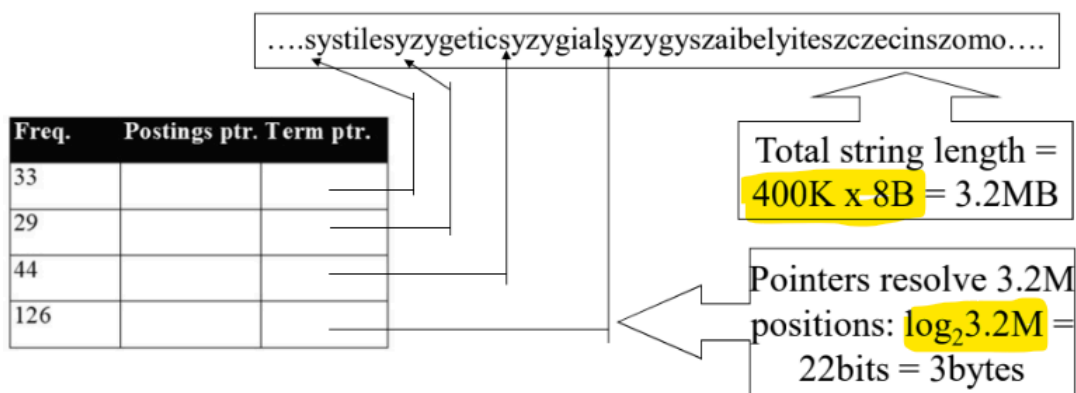
Number of positions = 3.2M

Bits required:

$$\log_2(3.2 \times 10^6)$$

$$\approx 22 \text{ bits}$$

22 bits $\approx$ 3 bytes instead of 20



◆ New Per-Term Storage

- 4 bytes → frequency
- 4 bytes → postings pointer
- 3 bytes → term pointer
- 8 bytes → average term length

Total:

$$4 + 4 + 3 + 8 = 19 \text{ bytes per term}$$

◆ Total Size Now

$$400,000 \times 19$$

$$= 7.6 \text{ MB}$$

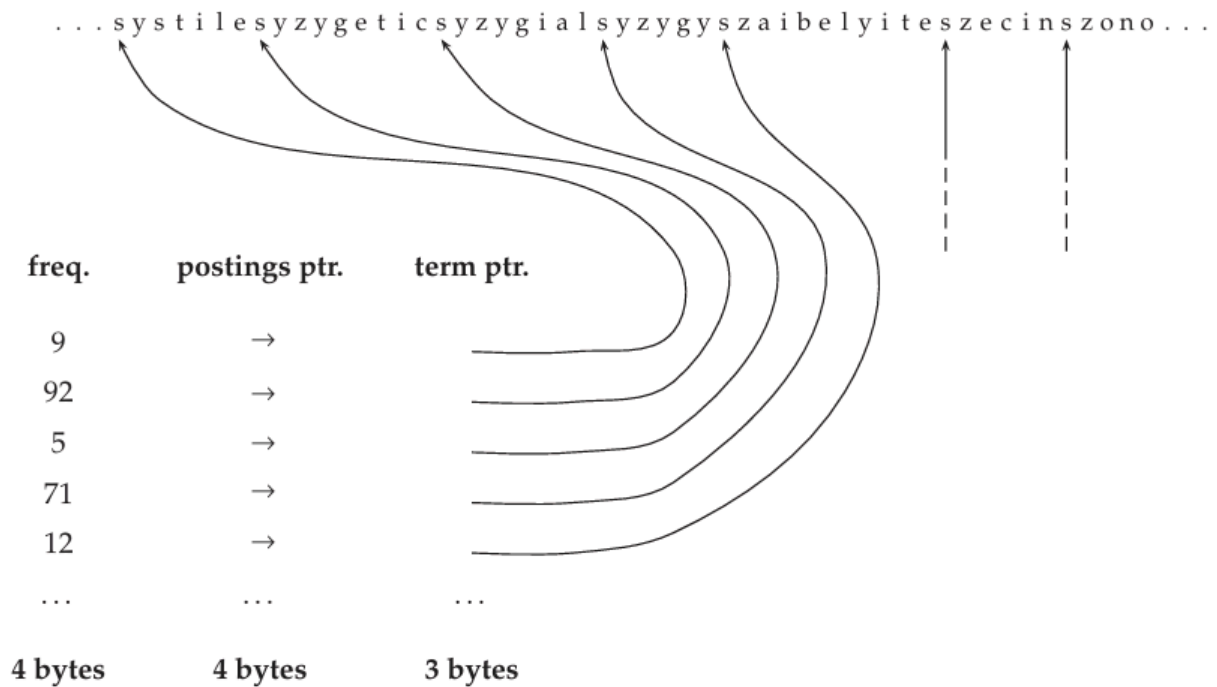


Figure 5.4 Dictionary-as-a-string storage. Pointers mark the end of the preceding term and the beginning of the next. For example, the first three terms in this example are systile, syzygetic, and syzygial.

★ Posting Compression

- Postings MUCHHH larger than dictionary [10× larger]
- the **dictionary** is typically **stored in memory (RAM)** for fast lookup, while the **posting lists are stored on disk**

- The **gaps between postings** are **short**.
- gaps are often **small numbers**, especially for **frequent words**.

We need:

- A **variable encoding method** that uses fewer bits for small gaps

Instead of storing:

⟨⟩ Code

5, 20, 25, 100

We store gaps:

⟨⟩ Code

5, 15, 5, 75

★ Variable length Encoding

Small numbers use fewer bits,
Large numbers use more bits.

Unlike fixed 32-bit integers, we don't waste space.

Each byte:

- **First bit** → **continuation bit** [MSB - leftmost]
 - **Last 7 bits** → **payload**
-
- If **continuation bit** = 0
→ **more bytes coming**
 - If **continuation bit** = 1
→ this is **last byte**

✓ Example 1: Encode 5

5 in binary = `101`

Fits in 7 bits → `0000101`

Since it's only one byte, it is the **last byte**, so MSB = 1:

VB(5):

Code

```
1 0000101
= 10000101
```

So 5 takes **1 byte**.

★ Gamma (γ) Encoding (Bit-Level)

- Better approach
- We're encoding **gaps** cause they're too small!

□ Example:

□ Binary code (732) = 1011011110

□ Gamma Code = # of Binary Digits (10) , Keeping the left most 1, they are nine

■ So, 0000000001 011011110

□ Decode {0000000001011011110} – takes nine zeros from start and take the next 10 binary digits, now get 1011011110

^ except the left most 1, there r 9 digits. So we append 9 zeroes on the left.

The 0s indicate that the next 10 digits r to be taken as **GAPS between the docs!!**

★ Index Compression

- Compressed inverted index supports **efficient Boolean retrieval**.
- Compressed index size $\approx$ **4% of total collection size**.
- Or $\approx$ **10–15% of the raw text size**.
- These savings assume only docIDs are stored (no positions).
- **Positional information** (for phrase/proximity queries) **increases index size**.
- In practice, indexes are larger, but compression techniques remain the same.

★ food for thoughts

11. Why compression is important for dictionary and posting list together? (short points)

- Dictionary is accessed on every query; compressing it helps keep it in RAM → faster lookups.
- Postings lists are the biggest part of the index (often 10× dictionary); compressing them saves the most disk space.
- Smaller postings = fewer disk reads / better cache fit → faster query processing.
- Compression lets search engines keep more of the “hot” index (dictionary + frequent postings) in memory.
- Overall: less storage cost + better latency + better scalability.

12. Given a dataset for the IR? How would you estimate the dictionary size and posting size? If it is known that it is a collection of English short stories, how would you justify the estimates?

How to estimate dictionary size (vocabulary M):

- Let T = total tokens in the collection (sum of all words after tokenization).
- Use Heap's law:
 - $M \approx kT^b$ where typically $b \approx 0.4-0.6$, $k \approx 30-100$ (depends on preprocessing).
- Estimated dictionary storage (rough):
 - Average term length ≈ 8 chars in English (before compression).
 - Plus per-term metadata: df (≈ 4 bytes) + postings pointer (≈ 4 bytes) (+ maybe term pointer if using “dictionary as a string”).
 - So per term $\approx$ (avg chars) + 8 bytes (+ pointer overhead).
 - Total dictionary bytes $\approx M \times$ (bytes/term).